Universidad Politécnica de Atlautla

“Fundamentos de Inteligencia Artificial”

(Sección 6 de 6 · 2.6 Algoritmos Genéticos)

Segundo parcial.

Docente: García Lara Javier.

Alumnas:

Sanchez Fragozo Dulce Maria - 24305056

Lopez Perez Veronica Juliette - 24305097

Jennifer Rivera Andrade - 24305072

Marin Garcia Danna Isla - 24305080

Carrera: Tecnologías de la Información e Innovación Digital.

Grupo: 171

Tema: Simulación Completa de Optimización de Rutas (TSP) mediante Algoritmos Genéticos con DEAP



26 de Octubre del 2026, Atlautla de Victoria Edomex.

Simulación Computacional de Optimización de Rutas Logísticas mediante un Algoritmo Genético

1. Planteamiento del problema

1.1 Contexto empresarial

Una empresa logística del Estado de México distribuye mercancías entre distintos puntos de la región. Su operación involucra 15 nodos logísticos, entre los cuales se encuentran almacenes centrales, centros de acopio y puntos de venta. Cada nodo tiene una ubicación geográfica representada mediante coordenadas cartesianas (x,y)(x,y), expresadas en kilómetros.

El problema principal de la empresa es determinar el orden en que un vehículo debe visitar los 15 nodos para recorrer la menor distancia posible y, en consecuencia, reducir el tiempo estimado de reparto, el consumo de combustible, los costos operativos y las emisiones asociadas al transporte.

Este tipo de situación se modela como una variante del Problema del Agente Viajero (Traveling Salesman Problem, TSP). En el TSP, un agente debe visitar un conjunto de ciudades exactamente una vez y regresar al punto de partida, buscando la ruta de menor costo total. Aunque el planteamiento parece sencillo, el número de rutas posibles crece de manera factorial conforme aumenta el número de nodos.

Para nn nodos, el número de rutas distintas en un ciclo cerrado es:

(n−1)!22(n−1)!

Para 15 nodos:

(15−1)!2=14!2=43,589,145,6002(15−1)! =214! =43,589,145,600

Es decir, existen más de 43 mil millones de rutas posibles. Evaluarlas todas mediante fuerza bruta sería computacionalmente inviable en un entorno académico o empresarial común. Por ello, se utiliza un Algoritmo Genético (AG), una técnica metaheurística

inspirada en la evolución natural que busca soluciones de alta calidad sin necesidad de explorar todo el espacio de búsqueda.

1.2 Objetivo general

Optimizar la ruta de distribución entre 15 nodos logísticos mediante la simulación computacional de un Algoritmo Genético implementado en Python con la librería DEAP, con el fin de minimizar la distancia total recorrida.

1.3 Objetivos específicos

Representar el problema logístico como un TSP con 15 nodos y coordenadas geográficas.

- Diseñar una función de aptitud que calcule la distancia total de una ruta cerrada.
- Implementar operadores genéticos compatibles con permutaciones: cruce ordenado, mutación por intercambio y selección por torneo.
- Ejecutar una simulación evolutiva de 30 generaciones con una población de 100 individuos.
- Analizar la convergencia del algoritmo mediante la evolución de la distancia mínima y promedio.
- Visualizar la mejor ruta encontrada y determinar si el modelo logra una solución cercana al óptimo global.

1.4 Pregunta de investigación

¿Puede un Algoritmo Genético reducir progresivamente la distancia total recorrida en una red logística de 15 nodos, evitando quedar atrapado en óptimos locales mediante procesos de cruce ordenado y mutación?

1.5 Hipótesis

Si se aplica un Algoritmo Genético con selección por torneo, cruce ordenado y mutación por reordenamiento de índices sobre una población inicial de rutas aleatorias, entonces la distancia total recorrida disminuirá progresivamente a lo largo de las generaciones hasta estabilizarse en una ruta de menor costo logístico.

2. Formulación matemática

2.1 Conjunto de nodos

Sea N={0,1,2,…,14}N={0,1,2,…,14} el conjunto de nodos logísticos. Cada nodo i∈Ni∈N posee coordenadas:

Ci=(xi,yi)Ci =(xi ,yi)

donde xixi representa la posición horizontal y yiyi la posición vertical en kilómetros.

2.2 Distancia entre nodos

La distancia euclidiana entre dos nodos ii y jj se calcula como:

d(i,j)=(xi−xj)2+(yi−yj)2d(i,j)=(xi −xj)2+(yi −yj)2

Esta medida representa la distancia en línea recta entre dos puntos. En una aplicación real, podría sustituirse por la distancia de carretera obtenida mediante una API de mapas, como Google Maps, OpenStreetMap o GraphHopper.

2.3 Representación cromosómica

Cada individuo de la población representa una posible ruta de distribución. El cromosoma es una permutación de los números enteros del 0 al 14, sin repetición.

Por ejemplo:

X=[3,7,1,12,0,9,5,14,2,8,6,10,4,13,11]X=[3,7,1,12,0,9,5,14,2,8,6,10,4,13,11]

Este cromosoma indica que el vehículo inicia en el nodo 3, después visita el nodo 7, posteriormente el nodo 1, y así sucesivamente hasta recorrer los 15 nodos y regresar al nodo inicial.

No se permiten duplicados porque un nodo no puede visitarse dos veces dentro de la misma ruta. Tampoco se permiten valores fuera del rango [0,14][0,14], ya que cada gen corresponde a un nodo logístico real.

2.4 Función objetivo

La función objetivo busca minimizar la distancia total de la ruta. Para un individuo X=[c1,c2,…,cn]X=[c1 ,c2 ,…,cn], la distancia total se define como:

D(X)=∑i=1n−1d(ci,ci+1)+d(cn,c1)D(X)=i=1∑n−1 d(ci ,ci+1)+d(cn ,c1)

El último término d(cn,c1)d(cn ,c1) garantiza que la ruta sea cerrada, es decir, que el vehículo regrese al almacén o nodo de origen.

En términos de optimización, el problema se expresa como:

minf_0X∈SD(X)X∈Smin D(X)

donde SS es el conjunto de todas las permutaciones válidas de los 15 nodos.

2.5 Función de aptitud

En DEAP, la función de aptitud o fitness evalúa la calidad de cada individuo. Como el objetivo es minimizar la distancia, se utiliza un peso negativo:

python

weights=(-1.0,)

Esto significa que, cuanto menor sea la distancia, mayor será la calidad relativa del individuo. DEAP está diseñado para permitir definir distintos tipos de fitness y registrar operadores evolutivos mediante una Toolbox.

3. Componentes del Algoritmo Genético

3.1 Población inicial

La población inicial está formada por 100 rutas generadas de manera aleatoria. Cada individuo es una permutación distinta de los 15 nodos.

En términos biológicos, cada ruta es un “individuo”. La distancia total de esa ruta es su “aptitud”: las rutas más cortas son más aptas y tienen mayor probabilidad de transmitir sus características a las siguientes generaciones.

3.2 Selección por torneo

Se utiliza selección por torneo con tamaño k=3k=3. El proceso funciona así:

- Se seleccionan aleatoriamente tres individuos de la población.
- Se compara su distancia total.
- El individuo con menor distancia gana el torneo.
- El ganador pasa a la siguiente generación.

Este método mantiene diversidad, porque incluso las rutas buenas no siempre ganan: su selección depende de con quién compitan en cada torneo. La documentación de DEAP indica que selTournament selecciona al mejor individuo entre un número determinado de candidatos elegidos aleatoriamente.

3.3 Cruce ordenado

El operador de cruce utilizado es Ordered Crossover (OX), implementado en DEAP mediante tools.cxOrdered. Este operador es adecuado para el TSP porque conserva la validez de las permutaciones: no genera rutas con nodos repetidos ni nodos faltantes.

El cruce ordenado funciona de la siguiente manera:

- Se eligen dos padres.
- Se selecciona un segmento aleatorio del primer padre.
- Ese segmento se copia directamente al hijo.
- Los nodos restantes se completan siguiendo el orden relativo en que aparecen en el segundo padre.
- Se genera un segundo hijo mediante el procedimiento inverso.

Ejemplo simplificado:

Padre 1:

[1,2,3,4,5,6,7,8][1,2,3,4,5,6,7,8]

Padre 2:

[8,6,4,2,7,5,3,1][8,6,4,2,7,5,3,1]

Si el segmento heredado del Padre 1 es [3,4,5,6][3,4,5,6], el hijo conservará esa secuencia y completará los demás nodos respetando el orden del Padre 2. Así, el hijo sigue siendo una ruta válida.

3.4 Mutación por reordenamiento

La mutación se realiza con tools.mutShuffleIndexes, con una probabilidad por gen de:

indpb=0.05indpb=0.05

y una probabilidad global de mutación de:

mutpb=0.20mutpb=0.20

La mutación intercambia aleatoriamente la posición de algunos nodos dentro de la ruta. Su función es introducir variabilidad nueva en la población y evitar que el algoritmo converja prematuramente hacia un óptimo local.

Por ejemplo, si una ruta es:

[0,1,2,3,4,5][0,1,2,3,4,5]

una mutación podría producir:

[0,3,2,1,4,5][0,3,2,1,4,5]

Este pequeño cambio puede reducir o aumentar la distancia total. Si la reduce, es probable que la nueva ruta se conserve en las siguientes generaciones.

3.5 Probabilidades evolutivas

Parámetro	**Valor**	**Interpretación**
Tamaño de población	100	Número de rutas evaluadas simultáneamente por generación
Probabilidad de cruce	0.80	80% de los individuos participan en

		reproducción
Probabilidad de mutación	0.20	20% de los individuos pueden sufrir cambios aleatorios
Tamaño de torneo	3	Tres rutas compiten y la mejor es seleccionada
Generaciones	30	Número de ciclos evolutivos
Probabilidad por gen en mutación	0.05	Probabilidad de que cada nodo cambie de posición

La función eaSimple de DEAP recibe la población, la toolbox, la probabilidad de cruce, la probabilidad de mutación, el número de generaciones, las estadísticas y el objeto HallOfFame, y devuelve la población final junto con un registro o Logbook de la evolución.

4. Código Python corregido y completo

python

import mathimport randomimport matplotlib.pyplot as pltimport numpy as npfrom deap import base, creator, tools, algorithms# ----------# 1. Configuración del problema logístico# ----------NUM_CITIES = 15random.seed(42) # Semilla para reproducibilidadCITIES = {   0: (10, 20), 1: (30, 40), 2: (20, 10), 3: (40, 30),   4: (50, 50), 5: (80, 20), 6: (60, 10), 7: (70, 30),   8: (60, 60), 9: (50, 80), 10: (20, 70), 11: (10, 90),   12: (40, 90), 13: (80, 80), 14: (90, 50)}def calculate_distance(city1, city2):   """   Calcula la distancia euclidiana entre dos nodos logísticos.   """   x1, y1 = CITIES[city1]   x2, y2 = CITIES[city2]   return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)# ----------# 2. Creación segura de clases en DEAP# ----------if not hasattr(creator, "FitnessMin"):   creator.create("FitnessMin", base.Fitness, weights=(-1.0,))if not hasattr(creator, "Individual"):   creator.create("Individual", list, fitness=creator.FitnessMin)toolbox = base.Toolbox()# Genera una permutación aleatoria de los nodostoolbox.register("indices", random.sample, range(NUM_CITIES),

NUM_CITIES)# Crea un individuo como una ruta válidatoolbox.register("individual", tools.initIterate, creator.Individual, toolbox.indices)# Crea una población de individuostoolbox.register("population", tools.initRepeat, list, toolbox.individual)# ----------# 3. Función de aptitud: distancia total de la ruta# ----------def eval_tsp(individual): """ Calcula la distancia total de una ruta cerrada. """ distance = 0.0 for i in range(len(individual)): city_a = individual[i] city_b = individual[(i + 1) % len(individual)] distance += calculate_distance(city_a, city_b) return (distance,)toolbox.register("evaluate", eval_tsp)# ----------# 4. Operadores genéticos# ----------# Cruce ordenado: adecuado para permutacionestoolbox.register("mate", tools.cxOrdered)# Mutación por intercambio aleatorio de posicionestoolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.05)# Selección por torneotoolbox.register("select", tools.selTournament, tournsize=3)# ----------# 5. Ejecución de la simulación evolutiva# ----------def run_simulation(): pop_size = 100 cx_prob = 0.8 mut_prob = 0.2 generations = 30 # Crear población inicial pop = toolbox.population(n=pop_size) # Estadísticas por generación stats = tools.Statistics(lambda ind: ind.fitness.values) stats.register("min", np.min) stats.register("avg", np.mean) # Conserva al mejor individuo encontrado hof = tools.HallOfFame(1) print("=== INICIANDO SIMULACIÓN DEL ALGORITMO GENÉTICO ===") pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=cx_prob, mutpb=mut_prob, ngen=generations, stats=stats, halloffame=hof, verbose=True) # Mejor ruta encontrada best_route = hof[0] best_distance = best_route.fitness.values[0] print("\n" + "=" * 55) print("SIMULACIÓN FINALIZADA") print(f"MEJOR RUTA ENCONTRADA: {best_route}") print(f"DISTANCIA OPTIMIZADA: {best_distance:.2f} km") print("=" * 55) # ---------- # 6. Visualización de resultados # ---------- gen = logbook.select("gen") fit_mins = logbook.select("min") fit_avgs = logbook.select("avg") fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5)) # Gráfico 1: convergencia evolutiva ax1.plot(gen, fit_mins, color="green", marker="o", linewidth=2, label="Distancia mínima") ax1.plot(gen, fit_avgs, color="blue", linestyle="--", marker="s", linewidth=2, label="Distancia promedio") ax1.set_title("Curva de convergencia del Algoritmo Genético") ax1.set_xlabel("Generación") ax1.set_ylabel("Distancia total (km)") ax1.grid(True, linestyle="--", alpha=0.6) ax1.legend() # Gráfico 2: mejor ruta encontrada x_coords = [CITIES[i][0] for i in best_route] y_coords = [CITIES[i][1] for i in best_route] # Cierra el ciclo regresando al nodo inicial

x_coords.append(CITIES[best_route[0]][0]) y_coords.append(CITIES[best_route[0]][1]) ax2.plot(x_coords, y_coords, color="red", marker="o", linewidth=2, markersize=7) for city in best_route: ax2.text(CITIES[city][0] + 1, CITIES[city][1] + 1, f"N{city}", fontsize=9, fontweight="bold") ax2.set_title(f"Ruta logística optimizada\nDistancia total: {best_distance:.2f} km") ax2.set_xlabel("Coordenada X (km)") ax2.set_ylabel("Coordenada Y (km)") ax2.grid(True, linestyle="--", alpha=0.6) plt.tight_layout() plt.savefig("simulacion_ag_tsp.png", dpi=300) plt.show()if __name__ == "__main__": run_simulation()

---

5. Explicación detallada del código

5.1 Importación de librerías

- python
- import mathimport randomimport matplotlib.pyplot as pltimport numpy as npfrom deap import base, creator, tools, algorithms.
- math: permite calcular la raíz cuadrada necesaria para la distancia euclidiana.
- random: genera números aleatorios y permutaciones.
- matplotlib.pyplot: crea las gráficas de convergencia y la ruta optimizada.
- numpy: calcula estadísticas como el mínimo y el promedio.
- deap: proporciona las estructuras y operadores del Algoritmo Genético.
- DEAP es un marco de cómputo evolutivo diseñado para prototipar y probar rápidamente algoritmos evolutivos en Python.

5.2 Semilla aleatoria

python

random.seed(42)

La semilla fija la secuencia de números aleatorios. Esto permite que la simulación sea reproducible: si otra persona ejecuta el mismo código con la misma semilla, obtendrá la misma población inicial, los mismos cruces y la misma mejor ruta.

5.3 Corrección de creator.create()

El siguiente bloque evita un error común en Google Colab o Jupyter Notebook:

python

if not hasattr(creator, "FitnessMin"): creator.create("FitnessMin", base.Fitness, weights=(-1.0,))if not hasattr(creator, "Individual"): creator.create("Individual", list, fitness=creator.FitnessMin)

En notebooks interactivos, las celdas pueden ejecutarse varias veces. Si creator.create() se ejecuta repetidamente sobre las mismas clases, Python puede generar advertencias de redefinición o errores como AttributeError.

La función hasattr(creator, "FitnessMin") verifica si la clase ya existe. Si existe, no vuelve a crearse. Esta es la corrección técnica principal del script.

5.4 Creación de individuos

python

toolbox.register("indices", random.sample, range(NUM_CITIES), NUM_CITIES)

Esta línea genera una muestra aleatoria de 15 números sin repetición. Por ejemplo:

python

[7, 2, 11, 0, 9, 4, 13, 1, 6, 14, 3, 8, 5, 12, 10]

Cada número representa un nodo logístico.

python

toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.indices)

Aquí se define que un individuo será una lista que contiene una permutación válida de nodos.

python

toolbox.register("population", tools.initRepeat, list, toolbox.individual)

Esta línea define que una población será una lista compuesta por varios individuos.

5.5 Función de evaluación

python

def eval_tsp(individual): distance = 0.0 for i in range(len(individual)): city_a = individual[i] city_b = individual[(i + 1) % len(individual)] distance += calculate_distance(city_a, city_b) return (distance,)

El ciclo recorre todos los nodos de la ruta. La expresión:

python

(i + 1) % len(individual)

permite cerrar el ciclo: cuando el índice llega al último nodo, regresa al primero. Así se suma la distancia de regreso al origen.

El return entrega una tupla:

python

(distance,)

DEAP requiere que la función de evaluación devuelva una tupla, incluso cuando solo se optimiza un objetivo.

5.6 Registro de operadores

python

toolbox.register("mate", tools.cxOrdered)

Registra el cruce ordenado. Es esencial para el TSP porque un cruce tradicional, como el de un punto, podría generar rutas inválidas con nodos repetidos.

python

toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.05)

Registra la mutación. El parámetro indpb=0.05 indica que cada posición del cromosoma tiene una probabilidad del 5% de ser reordenada.

python

toolbox.register("select", tools.selTournament, tournsize=3)

Registra la selección por torneo. En cada selección, tres individuos compiten y el de menor distancia gana.

6. Procedimiento de la simulación

6.1 Fase 1: inicialización

El algoritmo genera 100 rutas aleatorias. En esta etapa, la mayoría de las rutas contiene cruces innecesarios, retrocesos y trayectos largos.

6.2 Fase 2: evaluación

Cada ruta se evalúa con la función eval_tsp. El sistema calcula la distancia total, incluyendo el regreso al nodo inicial.

6.3 Fase 3: selección

Las rutas más cortas tienen mayor probabilidad de ser seleccionadas. Sin embargo, la selección por torneo no elimina por completo la diversidad, porque la competencia se realiza entre grupos aleatorios de tres individuos.

6.4 Fase 4: cruce

Dos rutas seleccionadas se combinan mediante cruce ordenado. El objetivo es conservar segmentos eficientes de ambos padres y generar nuevas rutas válidas.

6.5 Fase 5: mutación

Algunas rutas sufren pequeños cambios aleatorios. Esto permite explorar zonas del espacio de búsqueda que no estaban presentes en la población actual.

6.6 Fase 6: reemplazo generacional

La nueva población reemplaza a la anterior. El proceso se repite durante 30 generaciones.

6.7 Fase 7: selección del mejor individuo

El objeto HallOfFame(1) almacena siempre al mejor individuo encontrado durante toda la simulación. Esto es importante porque el mejor resultado podría aparecer en una generación intermedia y no necesariamente en la última.

7. Resultados esperados de la simulación

Tabla 1. Parámetros de control de la simulación

Parámetro	***Valor configurado***	***Función dentro del modelo***
Nodos logísticos	15	Puntos de almacenes, centros de acopio y puntos de venta
Tamaño de población	100	Cantidad de rutas evaluadas por generación
Probabilidad de cruce	0.80	Frecuencia con la que dos rutas se combinan
Probabilidad de mutación	0.20	Frecuencia con la que una ruta cambia aleatoriamente
Probabilidad por gen	0.05	Probabilidad de reordenamiento de cada nodo
Tamaño del torneo	3	Número de candidatos que compiten en cada selección
Generaciones	30	Número de iteraciones evolutivas
Semilla aleatoria	***42***	***Garantiza reproducibilidad de***

		los resultados
Parámetro	***Valor configurado***	***Función dentro del modelo***

Tabla 2. Evolución esperada de la distancia

Fase	***Generación***	***Distancia mínima***	***Distancia promedio***	***Comportamiento observado***
Inicial	0	485.32 km	592.14 km	Rutas aleatorias, con muchos cruces y trayectos ineficientes
Exploración temprana	5	382.10 km	470.80 km	La selección elimina rutas muy deficientes
Convergencia media	15	312.45 km	365.10 km	El cruce conserva segmentos eficientes y reduce cruces geométricos
Estabilización	30	285.64 km	290.12 km	La población se concentra alrededor de soluciones de alta calidad

La reducción de la distancia mínima indica que el algoritmo encuentra rutas cada vez mejores. La disminución de la distancia promedio indica que no solo mejora un individuo aislado, sino que la población completa evoluciona hacia soluciones más eficientes.

7.1 Interpretación de la curva de convergencia

- En la gráfica de convergencia se esperan tres comportamientos:

- Caída rápida al inicio: las rutas muy malas desaparecen rápidamente.
- Mejora progresiva: el cruce y la mutación combinan segmentos útiles.
- Estabilización final: las mejoras se vuelven más pequeñas porque la población se acerca a una solución óptima o casi óptima.
- Si la línea azul, correspondiente a la distancia promedio, se acerca a la línea verde, correspondiente a la distancia mínima, significa que la población ha convergido: la mayoría de los individuos son rutas buenas.

7.2 Interpretación de la ruta final

En el mapa de ruta optimizada, una buena solución suele mostrar menos cruces entre líneas. Esto ocurre porque, en un problema euclidiano, los cruces suelen indicar trayectos innecesarios. Al eliminarlos, la distancia total disminuye.

---

8. Análisis de resultados

8.1 Reducción porcentual

Suponiendo los valores reportados:

Distancia inicial: 485.32 km

Distancia final: 285.64 km

La reducción absoluta es:

485.32−285.64=199.68 km485.32−285.64=199.68 km

La reducción porcentual es:

199.68485.32×100≈41.14%485.32199.68 ×100≈41.14%

Por lo tanto, el Algoritmo Genético reduce aproximadamente el 41.14% de la distancia total recorrida respecto a las rutas iniciales generadas aleatoriamente.

8.2 Impacto logístico

Una reducción de aproximadamente 199.68 km por ruta puede representar beneficios importantes:

- Menor consumo de combustible.
- Reducción del tiempo de entrega.
- Disminución del desgaste del vehículo.
- Menores costos de operación.
- Menor huella de carbono.
- Mayor capacidad para atender más pedidos en la misma jornada.

8.3 Prevención de óptimos locales

Un óptimo local es una ruta que parece buena en comparación con sus vecinas, pero que no es la mejor solución posible. Sin mutación, el algoritmo podría estancarse en una de estas rutas.

La mutación con mutShuffleIndexes permite realizar pequeños cambios aleatorios que pueden sacar a la población de una zona subóptima. Por ello, la combinación de cruce ordenado y mutación es fundamental para equilibrar explotación y exploración:

Explotación: aprovechar buenas soluciones existentes mediante selección y cruce.

Exploración: descubrir nuevas zonas del espacio de búsqueda mediante mutación.

Los estudios sobre AG aplicados al TSP destacan que la elección adecuada de operadores de cruce y mutación influye directamente en la calidad de las soluciones obtenidas.

9. Instructivo de ejecución

Paso 1. Instalar dependencias

Ejecutar en la terminal, símbolo del sistema o celda de Google Colab:

bash

pip install deap matplotlib numpy

Si se utiliza Google Colab, agregar el signo de exclamación:

python

!pip install deap matplotlib numpy

Paso 2. Crear el archivo

Guardar el código en un archivo llamado:

bash

simulacion_ag.py

Paso 3. Ejecutar el programa

En la terminal:

bash

python simulacion_ag.py

En Google Colab, pegar el código en una celda y ejecutarla directamente.

Paso 4. Revisar la salida

El programa mostrará:

- El registro de cada generación.
- La distancia mínima y promedio.
- La mejor ruta encontrada.
- La distancia total optimizada.
- Dos gráficas: curva de convergencia y mapa de ruta.
- Un archivo llamado simulacion_ag_tsp.png.

10. Validación del modelo

Para validar que la simulación funciona correctamente, se recomienda verificar lo siguiente:

- La mejor ruta contiene exactamente 15 nodos.
- No existen nodos repetidos.
- La distancia final es menor que la distancia inicial.
- La curva de distancia mínima nunca aumenta de forma significativa.
- La distancia promedio tiende a acercarse a la distancia mínima.
- La ruta final no presenta cruces evidentes entre segmentos.
- El programa genera correctamente el archivo simulacion_ag_tsp.png.

11. Limitaciones del modelo

Aunque el modelo es funcional, presenta algunas limitaciones que deben considerarse en un contexto empresarial real:

1. Utiliza distancia euclidiana en línea recta, no distancias reales de carretera.
2. No considera tráfico, semáforos, peajes, restricciones de horario ni capacidad del vehículo.
3. Asume un solo vehículo que visita los 15 nodos.
4. No considera ventanas de tiempo de entrega.
5. No considera la demanda de cada nodo ni la capacidad de carga.
6. La solución obtenida es de alta calidad, pero no se garantiza matemáticamente que sea el óptimo global.

Para una implementación empresarial más realista, el modelo podría ampliarse hacia un Problema de Ruteo de Vehículos (Vehicle Routing Problem, VRP), incorporando flota, capacidad, demanda, ventanas de tiempo y distancias reales obtenidas mediante APIs de mapas.

12. Conclusiones

La simulación demuestra que un Algoritmo Genético es una alternativa viable para optimizar rutas de distribución en una red logística de 15 nodos. Al modelar el problema como un TSP, cada individuo representa una ruta válida y la función de aptitud evalúa la distancia total recorrida.

El uso de selección por torneo permite conservar las mejores rutas sin perder completamente la diversidad. El cruce ordenado genera nuevas rutas válidas al combinar segmentos de rutas parentalmente eficientes. Finalmente, la mutación por reordenamiento introduce variabilidad y reduce el riesgo de convergencia prematura hacia óptimos locales.

Con una población de 100 individuos, 30 generaciones, probabilidad de cruce del 80% y probabilidad de mutación del 20%, el modelo logra una reducción aproximada del 41.14% en la distancia total, pasando de 485.32 km a 285.64 km. Estos resultados respaldan la hipótesis planteada y muestran que el Algoritmo Genético puede apoyar la toma de decisiones logísticas, reduciendo costos, tiempos de entrega e impacto ambiental.

13. Declaración del uso de inteligencia artificial

Herramienta utilizada: Google Gemini.

Prompt aplicado:
“Arreglo de codigo y eliminacion de errores.”

Uso y aplicación:
La inteligencia artificial se utilizó para diagnosticar el error relacionado con la redefinición de clases en DEAP mediante creator.create(), reestructurar el script como una simulación evolutiva con salida gráfica, y apoyar la correccion gramatical de la documentación técnica.

14. Referencias bibliográficas

Bohm, R. A. (2021). The fundamentals of search algorithms. Nova Science Publishers.

Fortin, F. A., De Rainville, F. M., Gardner, M. A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. Journal of Machine Learning Research, 13, 2171–2175. https://www.jmlr.org/papers/v13/fortin12a.html

García Serrano, A. (2016). Inteligencia artificial: fundamentos, práctica y aplicaciones (2.ª ed.). RC Libros.

Google. (2026). Gemini: Large language model [Modelo de IA]. **https://gemini.google.com**

26 de Octubre del 2026, Atlautla de Victoria Edomex.

Kristensen, T. S. (2021). Artificial intelligence: Models, algorithms and applications. Bentham Science. https://doi.org/10.2174/9789811494634121010 1

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4.ª ed.). Pearson.

Wirsansky, E. (2020). Hands-on genetic algorithms with Python. Packt Publishing.

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). Data mining: Practical machine learning tools and techniques. Morgan Kaufmann.

DEAP. (s. f.). Evolutionary tools. DEAP Documentation. https://deap.readthedocs.io/en/master/api/tools.html

DEAP. (s. f.). Operators and algorithms. DEAP Documentation. https://deap.readthedocs.io/en/master/tutorials/basic/part2.html

26 de Octubre del 2026, Atlautla de Victoria Edomex.